

```
./run.sh --algo marl_multi --role multi --agents 4
```

```
algorithms/marl_multi/app.py
```

Agent $\times N$ — the algorithm

`transmit()`: the A2C actor decides *transmit* or *defer*
`on_result(ack)`: reward from its
own ack $\rightarrow$ actor + critic learn

`imports` `_shared/marl_learning.py`
`actor_MLP` · `critic_MLP`

N agents, adapted

```
run.sh · union/run_algo.py
```

load + adapt + choose the carrier

`load_app_factory()` imports `app.py` · `make(role)  $\times N$`
`adapt()`: `transmit/receive` $\rightarrow$ the uniform produce/consume
`build_channel()`: `--channel` picks the PHY

drives, slot by slot

```
union/phy_link.py
```

`run_slotted()` — the shared medium

each slot, all agents decide: 0 transmit $\rightarrow$ idle
· 1 $\rightarrow$ carried · $\geq 2 \rightarrow$ **collision, no ACK**
each agent hears only its own ack

the one winning burst

```
union/phy_link.py (make_channel)  $\rightarrow$  drivers/usrp · drivers/lora
```

PHY — `channel.transfer(burst)  $\rightarrow$  ack`

ack $\cdots$ *on_result()* $\rightarrow$ *next slot*